

Backend guide

Pizza Avenue V1 | End-to-end flow, data model, API inventory and acceptance tests

27	9	15	33
MSW routes	Specified-only paths	Requirement IDs	Test cases

IMPORTANT This checkout runs frontend apps and MSW mocks. There is no NestJS server, Prisma schema, migrated database or worker. Every planned behavior below is labeled Specified.

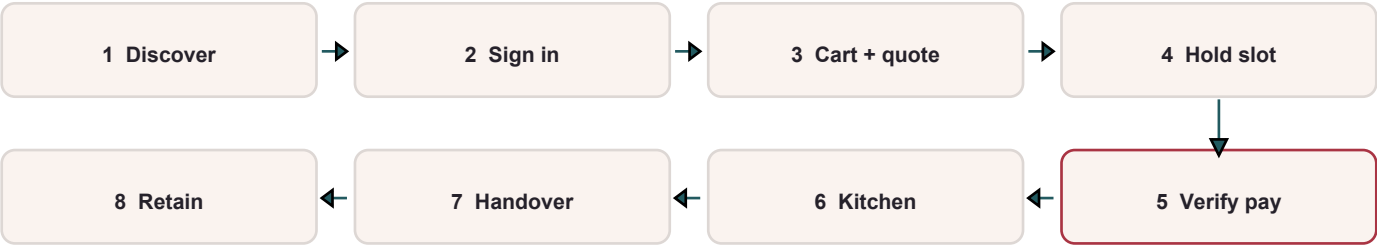
How to use this guide

Read	Use it for
Pages 2 onward: journey	Understand what happens from login through payment, kitchen, pickup and loyalty.
Architecture and schema	Find the intended backend boundaries and data ownership.
API inventory	Find all 27 mocked routes and 9 explicit contract-only paths.
Requirements and tests	Plan implementation and acceptance using linked IDs and current test status.

Status key

Label	Meaning
Implemented	Executable code exists in this checkout.
Mocked	MSW or in-memory fixture simulates a response; it is not server behavior.
Specified	Required by the project documents; no server implementation exists.
TODO: confirm	The source documents do not settle the detail.

Journey at a glance | Specified



Sources: docs/BACKEND.md | docs/16_DECISIONS.md | docs/08_DATA_MODEL.md | docs/10_API_CONTRACTS.md | docs/14_TEST_PLAN.md

01 The end-to-end order journey

A customer starts at a direct-entry surface, authenticates, configures a pickup order, pays, and receives a completed handover.

Step	Required backend action	Current checkout	Gate
1. Enter + identify	Phone OTP or authentic WhatsApp webhook -> verified identity -> revocable cookie session.	Auth is mocked; QR/webhook server routes absent.	AUTH-01
2. Build cart	Resolve live menu, variants and applicable modifiers; keep prices provisional.	Menu/cart fixtures and typed client.	MENU-01
3. Quote + reserve	Reprice in integer paise; validate promotions; hold pickup capacity safely.	Fixed quote; scenario-based slot response.	PRICE-01 / PICK-01
4. Start payment	Create idempotent attempt for server amount; preserve cart while pending.	Mock immediately returns a result.	PAY-01
5. Verify callback	Check provider signature and replay key; commit payment, order, capacity consumption and outbox event together.	No provider/webhook/transaction.	PAY-01 / REL-01
6. Prepare	Only verified paid CONFIRMED orders enter KDS; persist legal, authorized state changes.	Fixture queue; transition helper only.	ORDER-01
7. Handover	Counter validates code, blocks duplicate pickup, records actor and completion.	Contract only.	HAND-01 / OPS-01
8. Retain	Credit one loyalty ledger entry and Passport progress; reverse on confirmed refund.	Scenario balances/progress only.	LOY-01 / PASS-01

IMPORTANT The default payment mock returns **SUCCESS** and an order ID immediately. **V1** requires a verified provider webhook before confirmation and first KDS visibility.

Sources: docs/02_BUSINESS_RULES.md | docs/04_USER_FLOWS.md | docs/09_STATE_MACHINES.md | docs/16_DECISIONS.md | packages/mocks/src/handlers/payments.ts

02 Runtime architecture

Current code is a frontend workspace with a typed HTTP client and mock network boundary; the specified server is a modular monolith.

Current / Implemented + Mocked	Specified / Not built
Landing, Customer, KDS and Admin React/Vite apps	NestJS /api/v1 with REST/JSON/OpenAPI and live status transport
packages/api-client -> MSW handlers -> module fixtures	Prisma -> PostgreSQL business truth
Caddy and Compose package four static apps	Redis/BullMQ for short-lived work; PostgreSQL transactional outbox
API host routes to absent api:3000	Payment and WhatsApp adapters; API/worker/data services on private network

Authoritative boundary

Input	Server must decide	Recovery rule
Client cart, price, coupon	Availability, applicability, integer-paise total	Reject stale/invalid line and preserve viable cart.
Provider payment event	Authenticity, dedupe and final status	Replay may produce one order and one event only.
KDS socket event	Nothing authoritative; database state wins	Refetch API queue on reconnect or gap.
Completion event	One ledger earn and Passport progression	Consumer deduplicates after retry.

Sources: docs/16_DECISIONS.md | docs/21_DEPLOYMENT_ARCHITECTURE.md | compose.yaml | packages/api-client/src/client.ts | packages/mocks/src/handlers/

03 Identity, order and failure flows

These are the transaction and trust boundaries the backend must implement.

Flow	Sequence	Failure / replay behavior
Phone login	OTP request -> hashed challenge -> verify -> identity -> HttpOnly session	Cooldown, attempt limit, expiry, generic error; no raw OTP logging.
WhatsApp QR	QR attribution -> signed inbound webhook -> verified sender -> hashed one-use magic token -> session	QR text or ?phone= is never identity; forged or replayed provider event cannot log in.
Checkout	Quote -> capacity hold -> payment attempt -> signed callback -> atomic payment/order/hold/outbox	Failed/expired attempt releases hold; cart survives; KDS stays empty.
Operations	CONFIRMED -> PREPARING -> READY -> PICKED_UP -> COMPLETED	Actor and version checks; no illegal shortcut or second handover.
Retention	COMPLETED -> one loyalty earn + one Passport pair; refund -> new reversal entry	Retry does not duplicate earn, milestone or refund effect.

Sources: docs/02_BUSINESS_RULES.md | docs/09_STATE_MACHINES.md | docs/11_INTEGRATIONS.md | docs/18_ACCEPTANCE_CRITERIA.md

04 Database design (specified, not migrated)

The model below comes from the data-model specification. No physical table, index or migration can be confirmed from this checkout.



Selected logical relationships only; all entities are specified and unmigrated.

Domain	Specified entities
Identity	users, auth_identities, customer_profiles, staff_users, sessions, magic_login_tokens, otp_challenges, roles / permissions / user_roles / role_permissions
Store	stores, store_hours, store_overrides, pickup_capacity_rules, pickup_reservations
Menu	categories, products, product_variants, modifier_groups, modifiers, modifier_applicability, availability_overrides
Cart	carts, cart_items, cart_item_modifiers
Orders	orders, order_items, order_item_modifiers, order_events
Payments	payment_attempts, payment_transactions, refunds, webhook_events
Pickup	handover_events
Loyalty	loyalty_accounts, loyalty_transactions, rewards, reward_redemptions, passport_programs, passport_items, customer_passport_progress
Growth	promotions, promotion_rules, promotion_redemptions, referral_codes, referral_events, upsell_rules, qr_sources
Messaging	notification_preferences, notifications, notification_attempts
Platform	audit_events, integration_events, outbox_events, analytics_events

Key fields in the specified model

Entity	Specified fields (selected)
users	id, status, created_at
auth_identities	user_id, provider, provider_identifier, verified_at
sessions	user_id, session token hash/opaque ID, auth method, expiry, revoked_at
pickup_reservations	cart/order, slot, reserved units, expires_at, status
orders	public number, customer, store, source, state, pickup type, subtotal/discount/tax/total
order_items	product/variant name snapshot, unit price, quantity, line total
payment_attempts	provider, reference, amount, status, idempotency_key
webhook_events	provider, provider_event_id, authenticity, processing status
outbox_events	event/aggregate, payload/schema version, state, idempotency key, retry fields
loyalty_transactions	points_delta, type, source, source_id

Critical relationships and constraints

Relationship / entity	Constraint or invariant
users -> auth_identities	Unique (provider, provider_identifier); PHONE and WHATSAPP may resolve to one user.
orders -> order_items -> modifiers	Purchased name, variant and integer-paise price are immutable snapshots.
payment_attempts / transactions / webhook_events	Scoped idempotency key; unique provider transaction and (provider, provider_event_id) where supplied.
pickup_reservations	Store/slot capacity locking; HELD -> CONSUMED, RELEASED or EXPIRED.
orders -> order_events	Append-only order-state history; conditional version transitions.
outbox_events	Insert with critical domain update in the same PostgreSQL transaction; retryable consumer key.
loyalty_transactions / passport progress	One qualifying earn and one order/product pair; reversal is a new ledger entry.

IMPORTANT These are logical requirements, not a generated Prisma schema. Exact field types, foreign keys and physical indexes remain TODO: confirm during migration design.

Sources: docs/08_DATA_MODEL.md | docs/16_DECISIONS.md | docs/BACKEND.md

05 API inventory

Every route uses /api/v1. Auth column shows mock enforcement / required audience. Request and response are compact examples from the working reference.

Authentication | Mocked

Method + path	Auth	Request	Response	Errors
POST /auth/otp/request	none / public	{"phone":"+919000000001"}	{"accepted":true}	None
POST /auth/otp/verify	none / public	{"phone":"+919000000001","otp":"123456"}	{"customer":{"id":"customer-arjun"},"session":{"id":"mock-session","authMethod":"PHONE_OTP"}} (excerpt)	None
POST /auth/magic/consume	none / public	{"token":"example-one-time-token"}	{"customer":{"id":"customer-arjun"},"session":{"id":"mock-session","authMethod":"WHAT_SAPP_QR_MAGIC_LINK"}} (excerpt)	None
POST /auth/logout	none / current session	No body	HTTP 204, empty body	None

Menu | Mocked

Method + path	Auth	Request	Response	Errors
GET /stores/{storeId}/menu	none / public	/stores/sainikpuri/menu	{ "storeId": "sainikpuri", "version": "mock-menu-1" } (excerpt)	None
GET /products/{productId}	none / public	/products/pizza-avenue-signature	{ "id": "pizza-avenue-signature", "name": "Avenue Signature" } (excerpt)	404 NOT_FOUND

Cart and quote | Mocked

Method + path	Auth	Request	Response	Errors
POST /carts	none / customer or guest session	{ "storeId": "sainikpuri" }	{ "id": "cart-mock-1", "status": "ACTIVE", "items": [] } (excerpt)	None
GET /carts/{cartId}	none / cart owner	/carts/cart-mock-1	{ "id": "cart-mock-1", "items": [] } (excerpt)	None
POST /carts/{cartId}/items	none / cart owner	{ "productId": "pizza-diavola", "variantId": "pizza-diavola-regular", "quantity": 1, "selectedModifiers": [], "productNameSnapshot": "Diavola", "variantNameSnapshot": "Regular", "notes": null, "provisionalUnitPrice": { "amount": 44900, "currency": "INR" } }	{ "id": "cart-mock-1", "items": [{ "id": "cart-item-1" }] } (excerpt)	None
PATCH /carts/{cartId}/items/{itemId}	none / cart owner	{ "quantity": 2 }	{ "id": "cart-mock-1", "items": [{ "id": "cart-item-1", "quantity": 2 }] } (excerpt)	None
DELETE /carts/{cartId}/items/{itemId}	none / cart owner	/carts/cart-mock-1/items/cart-item-1	{ "id": "cart-mock-1", "items": [] } (excerpt)	None
POST /carts/{cartId}/quote	none / cart owner	No body	{ "cartId": "cart-mock-1", "payableTotal": { "amount": 44900, "currency": "INR", "version": "quote-1" } } (excerpt)	None

Pickup | Mocked

Method + path	Auth	Request	Response	Errors
GET /stores/{storeId}/pickup-options	none / public	/stores/sainikpuri/pickup-options	{ "asap": { "id": "slot-asap", "state": "AVAILABLE", "scheduled": [{ "id": "slot-1430" }] } } (excerpt)	None
POST /pickup/reservations	none / customer or guest session	{ "cartId": "cart-mock-1", "slotId": "slot-asap", "pickupType": "ASAP", "reservedCapacityUnits": 1 }	201 { "id": "reservation-mock-1", "status": "HELD", "cartId": "cart-mock-1" } (excerpt)	409 SLOT_FULL

Payments | Mocked

Method + path	Auth	Request	Response	Errors
POST /payments	none / customer	Header Idempotency-Key: example-key; { "cartId": "cart-mock-1" }	201 { "id": "payment-mock-1", "status": "SUCCESS", "orderId": "order-pa-1001" } (excerpt)	None
GET /payments/{paymentId}	none / payment owner	/payments/payment-mock-1	{ "id": "payment-mock-1", "status": "SUCCESS" } (excerpt)	None

Orders and kitchen | Mocked

Method + path	Auth	Request	Response	Errors
GET /orders/me	none / customer	No body	{ "items": [{ "id": "order-pa-1001" }], "page": 1, "page Size": 5, "total": 5 } (excerpt)	None
GET /orders/{orderId}	none / owner or authorized staff	/orders/order-pa-1001	{ "id": "order-pa-1001", "status": "CONFIRMED", "version": 1 } (excerpt)	404 NOT_FOUND
POST /orders/{orderId}/reorder	none / order owner	No body	{ "cartId": "cart-reorder-1" }	None
GET /kds/orders	none / Kitchen, Manager, Founder	No body	[{ "id": "order-pa-1001", "status": "CONFIRMED" }] (excerpt)	None
POST /orders/{orderId}/preparing	none / Kitchen, Manager, Founder	No body	{ "id": "order-pa-1001", "status": "PREPARING", "version": 2 } (excerpt)	409 CONFLICT for unknown ID; illegal transition throws
POST /orders/{orderId}/ready	none / Kitchen, Manager, Founder	No body	{ "id": "order-pa-1002", "status": "READY", "version": 2 } (excerpt)	409 CONFLICT for unknown ID; illegal transition throws

Loyalty and Passport | Mocked

Method + path	Auth	Request	Response	Errors
GET /me/loyalty	none / customer	No body	{ "id": "loyalty-arjun", "pointsBalance": 840, "status": "ACTIVE" } (excerpt)	None
GET /me/rewards	none / customer	No body	[{ "id": "reward-dip", "name": "Free dip", "pointsCost": 300, "active": true }] (excerpt)	None
POST /rewards/{rewardId}/reserve	none / customer	No body	{ "id": "redemption-mock-1", "rewardId": "reward-dip", "status": "RESERVED" } (excerpt)	None
GET /me/passport	none / customer	No body	{ "programId": "passport-signatures", "completed": false, "unlockedRewardId": null } (excerpt)	None

Admin | Mocked

Method + path	Auth	Request	Response	Errors
PUT /admin/store-state	none / change_store_state	{ "storeId": "sainikpuri", "state": "PAUSED" }	{ "id": "sainikpuri", "state": "PAUSED" } (excerpt)	None

Mocked route source: packages/mocks/src/handlers/ and packages/api-client/src/. Full typed shapes and source links: docs/BACKEND.md section 6.

Named contract paths without handlers | Specified

Method + path	Auth	Request	Response / unknowns
GET /qr/{code}	Public	/qr/SAINIKPURI_COUNTER	Redirect to official WhatsApp deep link; Fallback status/location: TODO: confirm
POST /webhooks/whatsapp	Verified provider signature	Provider-signed request; shape TODO: confirm	Duplicate-safe acknowledgement; shape TODO: confirm; Signature failure/status: TODO: confirm
POST /webhooks/payments/{provider}	Verified provider signature	Provider-signed request; shape TODO: confirm	Acknowledgement: TODO: confirm; Signature/replay status: TODO: confirm
POST /payments/{id}/refund	Authorized refund actor	TODO: confirm	Pending/confirmed refund shape TODO: confirm; Eligibility codes TODO: confirm
GET /counter/ready-orders	Counter, Manager, Founder	No body	Ready-order collection shape TODO: confirm; TODO: confirm
POST /orders/{id}/pickup	Counter, Manager, Founder	TODO: confirm	Picked-up order shape TODO: confirm; Duplicate handover code TODO: confirm
DELETE /reward-reservations/{id}	Customer owner	No body	Release response TODO: confirm; TODO: confirm
GET /health	Public or internal: TODO: confirm	No body	Liveness payload TODO: confirm; TODO: confirm
GET /health/ready	Public or internal: TODO: confirm	No body	Readiness payload TODO: confirm; TODO: confirm

Sources: docs/10_API_CONTRACTS.md | docs/BACKEND.md section 6

IMPORTANT GET /carts/{cartId} and GET /payments/{paymentId} exist in client/mocks but are absent from the named API contract. Pickup error code also differs: PICKUP_SLOT_FULL in one contract example versus SLOT_FULL in pickup contract and mock.

06 Permissions and security

The roles are a specified server policy. Current MSW handlers do not check sessions, resource ownership or staff roles.

Actor	Order access	Operational actions	Commercial actions
Customer	Own orders	Own cart and reward flow	No staff or store configuration
Kitchen	KDS queue	Prepare and mark ready	No price, refund or store configuration
Counter	Ready/pickup queue	Verify handover	No price or refund by default
Manager	Store-scoped operations	Operational actions in scope	Policy-specific permissions: TODO: confirm
Founder	All operational views	Admin and audit oversight	Sensitive commercial actions with audit

Backend release checks

Control	Required evidence
Authentication	Hashed expiring OTP and magic token; authentic provider webhook; revocable Secure/HttpOnly cookie.
Authorization	Server-side permission and ownership checks on every protected API.
Payment	Provider signature, event dedupe, operation idempotency and paid-only KDS.
Input / money	Validated request data and safe integer paise; never trust client totals.
Infrastructure	Private PostgreSQL/Redis, CSRF and CORS policy, rate limits, redacted logs, encrypted off-server backup and restore test.

Sources: docs/03_ROLES_PERMISSIONS.md | docs/16_DECISIONS.md | docs/21_DEPLOYMENT_ARCHITECTURE.md

07 Requirement register

Each ID is tied to a source and test family; statuses describe the current checkout, not delivery readiness.

ID	Requirement	Current status	Tests
AUTH-01	OTP and verified WhatsApp login resolve one customer and revocable session.	Mocked / Specified	T-AUTH-01-05
MENU-01	Checkout rejects unavailable items and invalid variant/modifier combinations.	Specified; fixture sold-out state only	T-MENU-01-02
PRICE-01	Server quotes all money in integer paise; client totals cannot override payable amount.	Implemented types/helper, Mocked quote	T-PRICE-01-02
PICK-01	Store-scoped capacity units and holds prevent overbooking; payment consumes hold.	Mocked slots, Specified locking	T-PICK-01-03
PAY-01	Verified signed webhook confirms payment/order once, consumes capacity and inserts outbox event atomically.	Specified; immediate mock result conflicts	T-PAY-01-04
ORDER-01	Legal order transitions, append-only events and first KDS visibility at CONFIRMED.	Implemented transition helper, Mocked KDS list	T-ORDER-01-03
HAND-01	Verified code handover blocks duplicates; COMPLETED drives downstream earn/progress.	Specified	T-HAND-01-02
LOY-01	Ledger earns once after completed order; refund reverses through a new transaction.	Mocked balance, Specified ledger	T-LOY-01-02
REWARD-01	Server validates and reserves reward; failure releases it.	Mocked reserve, Specified lifecycle	T-REWARD-01
PASS-01	Only qualifying completed orders advance Passport once per order/product pair.	Mocked progress, Specified consumer	T-PASS-01
REFUND-01	Authorized, auditable refunds wait for provider confirmation and adjust loyalty.	Specified	T-REFUND-01
REORDER-01	Historical order is intent; map to current menu/price and surface omissions.	Mocked fixed cart ID	T-REORDER-01
OPS-01	Sensitive staff actions enforce role/store scope and audit.	Specified	T-SEC-01-02
REL-01	Domain change and outbox commit together; retries, stale leases and consumers are idempotent.	Specified	T-REL-01-02
DEP-01	Private data services, health checks, off-server backups and restore test.	Specified; current Compose is frontend only	T-DEP-01-02

Sources: docs/BACKEND.md section 8.1; each row there links to its project source

08 Test plan and acceptance

The 33 cases below connect the required behavior to executable evidence or an explicit implementation blocker.

4	2	27
Passing mock/helper cases	Needed current-code cases	Blocked until backend exists

Current local suite: 6 files and 37 tests passed on 10 Oct 2026. Coverage reporting is not configured. A passing mock case is not backend acceptance.

ID / req	Category	Scenario + input	Expected	Status
T-AUTH-01 / AUTH-01	API/security	Invalid, expired, replayed or rate-limited OTP Phone and OTP attempts	No session; generic response; limits enforced	Blocked: backend absent

ID / req	Category	Scenario + input	Expected	Status
T-AUTH-02 / AUTH-01	Integration/security	Active/inactive QR and authentic/forged/duplicate WhatsApp events QR code; signed or forged webhook	Only verified sender creates one identity/token; source attributed	Blocked: backend absent
T-AUTH-03 / AUTH-01	Concurrency/security	Expired, used and concurrent magic consume Same token twice/concurrently	One session at most; generic recovery	Blocked: backend absent
T-AUTH-04 / AUTH-01	API/security	Session cookie, CSRF, logout, revocation, origin Mutations with/without valid session and CSRF	Secure/HttpOnly/SameSite cookie; invalid requests rejected	Blocked: backend absent
T-AUTH-05 / AUTH-01	Data/concurrency	PHONE and WHATSAPP verified phone match Two concurrent identity resolutions	One user; ambiguous link sent to support review	Blocked: backend absent
T-MENU-01 / MENU-01	Client/mock	Menu fixture and pause currency load getMenu('sainikpuri')	Product present; base price currency INR	Passing
T-MENU-02 / MENU-01	API/domain	Sold-out item or invalid modifier at checkout Item becomes unavailable after cart creation	Reject affected line; preserve rest of cart	Blocked: backend absent
T-PRICE-01 / PRICE-01	Unit	Unsafe/fractional pause inrFromPause(1.5) or unsafe integer	Reject value	Needed
T-PRICE-02 / PRICE-01	API/security	Client tampers with price/discount Cart/payment body with lower total	Server calculates and charges authoritative amount	Blocked: backend absent
T-PICK-01 / PICK-01	Client/mock	Full-slot options scenario setScenario('PICKUP_FULL'); fetch options	ASAP/scheduled slots report FULL	Passing
T-PICK-02 / PICK-01	Mock/API	Reservation during full scenario POST /pickup/reservations after PICKUP_FULL	409 SLOT_FULL envelope	Needed
T-PICK-03 / PICK-01	Data/concurrency	Last-slot race and hold lifecycle Concurrent holds, expiry and verified payment	No overbooking; exactly one consumption	Blocked: backend absent
T-PAY-01 / PAY-01	Client/mock	Payment failure scenario createPaymentAttempt after PAYMENT_FAILURE	FAILED, orderId: null	Passing
T-PAY-02 / PAY-01	Integration/security	Invalid signature or client success spoof Forged callback/client redirect	No confirmed order or KDS ticket	Blocked: backend absent
T-PAY-03 / PAY-01	Data/idempotency	Duplicate/replayed successful callback Same provider event/transaction repeatedly	One payment, order, hold consumption and first KDS event	Blocked: backend absent
T-PAY-04 / PAY-01	E2E	Failed/expired payment and retry Failed attempt, then retry	Cart survives; no KDS before verified success	Blocked: backend absent
T-ORDER-01 / ORDER-01	Unit	Legal path and illegal shortcut CONFIRMED -> PREPARING; DRAFT -> READY	First allowed; second throws	Passing
T-ORDER-02 / ORDER-01	API/concurrency	Stale or unauthorized KDS action Two actors update same version	One legal transition; stale action gets conflict	Blocked: backend absent
T-ORDER-03 / ORDER-01	Reliability	Paid-only KDS and socket reconnect Unpaid order; event gap/disconnect	Unpaid absent; API refetch restores truth	Blocked: backend absent
T-HAND-01 / HAND-01	E2E	Ready order handover Valid customer code	PICKED_UP, then COMPLETED with event	Blocked: backend absent
T-HAND-02 / HAND-01	API/security	Duplicate or wrong-code handover Reuse code / second pickup	Reject and audit; no second completion	Blocked: backend absent
T-LOY-01 / LOY-01	Data/idempotency	Completion event replay Same completed order twice	One ledger earn and stable derived balance	Blocked: backend absent
T-LOY-02 / LOY-01	Data/integration	Confirmed partial/full refund Provider confirmation	New reversal ledger entry; history retained	Blocked: backend absent
T-REWARD-01 / REWARD-01	API/domain	Ineligible reserve and failed payment Insufficient points; payment fails after hold	Reject ineligible reward; release failed hold	Blocked: backend absent
T-PASS-01 / PASS-01	Data/idempotency	Qualifying pair replay Same product/order completion twice	Progress advances once; milestone once	Blocked: backend absent
T-REFUND-01 / REFUND-01	Integration/RBAC	Unauthorized refund and provider outage Kitchen actor; provider timeout	Deny actor; authorized refund remains pending/retryable	Blocked: backend absent
T-REORDER-01 / REORDER-01	API/domain	Old item or price changed Historical order with discontinued modifier	Current menu mapping and price; changes shown for confirmation	Blocked: backend absent

ID / req	Category	Scenario + input	Expected	Status
T-SEC-01 / OPS-01	RBAC	Cross-customer/store access Customer reads another order; Kitchen edits price	Server rejects both	Blocked: backend absent
T-SEC-02 / OPS-01	Audit/logging	Sensitive action and credential redaction Refund/store-state change; OTP/token in request	Actor audit exists; no raw secret in logs	Blocked: backend absent
T-REL-01 / REL-01	Data/recovery	Crash after domain commit before publish Kill worker between transaction and delivery	Outbox row persists and is retryable	Blocked: backend absent
T-REL-02 / REL-01	Data/recovery	Replay, stale lease and exhausted retries Retry consumer/reclaim stale event	One effect; failed event visible to operator	Blocked: backend absent
T-DEP-01 / DEP-01	Infrastructure	Private data services and readiness Inspect Compose ports; call health routes	DB/Redis private; liveness/readiness reflect dependencies	Blocked: backend absent
T-DEP-02 / DEP-01	Recovery	Off-server backup and isolated restore Encrypted dump/upload/restore exercise	Restored data verified; retention/alerting visible	Blocked: backend absent

Sources: docs/BACKEND.md section 12 | docs/14_TEST_PLAN.md | docs/18_ACCEPTANCE_CRITERIA.md

09 Deployment, integrations and operational readiness

Current Compose and CI cover static apps. API, worker, data services, provider connections and recovery are specified.

Area	Current state	Required production gate
CI/CD	Install, lint, typecheck, test and build.	Backend/domain tests, migration review, staged deploy and rollback.
Compose/Caddy	Four frontend services and edge router; API upstream absent.	API, worker, PostgreSQL and Redis with private data ports.
Payment	Scenario-only MSW result.	Signed callbacks, idempotency, reconciliation and safe failure.
WhatsApp	Magic consume mock only.	Authentic webhook, sender verification and retrievable outbound reply.
Live status	No socket server.	Refetch authoritative queue after reconnect/gap.
Monitoring	No backend logs or health implementation.	Structured redacted logs, health/readiness, webhook/worker alarms.
Backups	No DB to back up.	Encrypted off-server backup and isolated restore exercise.

Build order

Priority	Work	Acceptance tests
P0	NestJS + Prisma/PostgreSQL foundation, session/RBAC, authoritative price and capacity.	T-AUTH-01-05, T-MENU-02, T-PRICE-02, T-PICK-03, T-SEC-01
P0	Verified payment callback, atomic order/hold/outbox and paid-only KDS.	T-PAY-02-04, T-ORDER-03, T-REL-01
P1	Conditional KDS transitions, handover, loyalty, Passport and refund lifecycle.	T-ORDER-02, T-HAND-01-02, T-LOY-01-02, T-PASS-01, T-REFUND-01
P1	Worker retries, health, audit/logging, backup and restore.	T-REL-02, T-SEC-02, T-DEP-01-02

IMPORTANT Open decisions: canonical pickup error code, manager permission details, provider selections, KPI targets and pagination parameters remain TODO: confirm. No secrets or backend environment variables are asserted as active.

Sources: docs/16_DECISIONS.md | docs/21_DEPLOYMENT_ARCHITECTURE.md | docs/BACKEND.md sections 13-16